

测试目标

本文档用于压力测试 PDF 渲染链路：标题层级、段落、行内公式、块级公式、多行公式、矩阵、表格中的公式、代码块高亮、长代码换行、Callout、分页和目录书签。每一节之间插入显式分页，目标输出约 10 页。

第 1 页：基础数学排版与 TypeScript

这段文字用于观察行内公式和中文混排。样本均值记为 $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ ，若 $X_1, X_2, \dots, X_n$ 独立同分布且 $E(X_i) = \mu$ 、 $D(X_i) = \sigma^2$ ，则有 $E(\bar{X}) = \mu$ 、 $D(\bar{X}) = \frac{\sigma^2}{n}$ 。

$$\begin{aligned} E(\bar{X}) &= E\left(\frac{1}{n} \sum_{i=1}^n X_i\right) \\ &= \frac{1}{n} \sum_{i=1}^n E(X_i) \\ &= \mu. \end{aligned}$$

项目	公式	说明
样本均值	$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$	行内分式与求和
样本方差	$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$	检查上下标
标准化	$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}}$	检查根号和分式

```
export type TaskStatus = 'queued' | 'running' | 'succeeded' | 'failed';

export interface WorkflowTask {
  id: string;
  title: string;
  status: TaskStatus;
  retryCount: number;
  createdAt: string;
  updatedAt: string;
}

export function nextStatus(task: WorkflowTask, ok: boolean): TaskStatus {
  if (task.status === 'succeeded') return 'succeeded';
  if (ok) return 'succeeded';
  return task.retryCount >= 3 ? 'failed' : 'queued';
}
```

第 2 页：极限、泰勒展开与 Python

常见极限测试行内公式： $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$, $\lim_{x \rightarrow 0} \frac{1 - \cos x}{x^2} = \frac{1}{2}$, $\lim_{n \rightarrow \infty} (1 + \frac{1}{n})^n = e$.

$$\begin{aligned}e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + O(x^4), \\ \sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} + O(x^7), \\ \ln(1 + x) &= x - \frac{x^2}{2} + \frac{x^3}{3} + O(x^4).\end{aligned}$$

观察点

这里要看行内的极限上下标是否被压得过低，块级多行公式是否居中，阶乘、上标和 $O(x^n)$ 是否清晰。

```
def taylor_exp(x: float, terms: int = 8) -> float:
    """Approximate exp(x) by a Taylor polynomial."""
    total = 0.0
    factorial = 1.0
    power = 1.0
    for k in range(terms):
        if k > 0:
            power *= x
            factorial *= k
        total += power / factorial
    return total

for value in [0.1, 0.5, 1.0]:
    print(value, taylor_exp(value), 'error=', abs(taylor_exp(value) -
__import__('math').exp(value)))
```

第3页：级数、收敛性与 C++

正项级数常用比较判别法、比值判别法和根值判别法。若 $a_n > 0$ ，且

$$\rho = \lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n},$$

则 $\rho < 1$ 时级数 $\sum_{n=1}^{\infty} a_n$ 收敛， $\rho > 1$ 时发散， $\rho = 1$ 时不确定。

交错级数示例：

$$\sum_{n=1}^{\infty} (-1)^{n-1} \frac{1}{n} = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots = \ln 2.$$

```
#include <cmath>
#include <iostream>
#include <vector>

double alternating_harmonic(int n) {
    double sum = 0.0;
    for (int k = 1; k <= n; ++k) {
        double term = 1.0 / k;
        if (k % 2 == 0) term = -term;
        sum += term;
    }
    return sum;
}

int main() {
    for (int n : {10, 100, 1000, 10000}) {
        std::cout << n << " -> " << alternating_harmonic(n)
                    << " error=" << std::abs(alternating_harmonic(n) -
std::log(2.0)) << '\n';
    }
}
```

第 4 页：多元函数、梯度与 Rust

设二元函数 $f(x, y) = x^2 + xy + 2y^2$ ，其梯度和 Hessian 矩阵为

$$\nabla f(x, y) = \begin{pmatrix} 2x + y \\ x + 4y \end{pmatrix}, \quad H_f = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}.$$

若 H_f 正定，则 f 是严格凸函数。由顺序主子式 $\Delta_1 = 2 > 0$ 、 $\Delta_2 = 7 > 0$ ，可知 H_f 正定。

```
#[derive(Debug, Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

fn gradient(p: Point) -> Point {
    Point {
        x: 2.0 * p.x + p.y,
        y: p.x + 4.0 * p.y,
    }
}

fn step(p: Point, lr: f64) -> Point {
    let g = gradient(p);
    Point { x: p.x - lr * g.x, y: p.y - lr * g.y }
}
```

第 5 页：线性代数、矩阵与 SQL

矩阵公式用于测试大括号、行列式、矩阵、转置和特征值排版。

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \quad \det(A - \lambda I) = \begin{vmatrix} 2 - \lambda & -1 & 0 \\ -1 & 2 - \lambda & -1 \\ 0 & -1 & 2 - \lambda \end{vmatrix}.$$

二次型：

$$q(x_1, x_2, x_3) = 2x_1^2 + 2x_2^2 + 2x_3^2 - 2x_1x_2 - 2x_2x_3 = x^T Ax.$$

```
create table matrix_cases (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  dimension integer not null check (dimension > 0),  
  matrix jsonb not null,  
  determinant numeric,  
  created_at timestamptz not null default now()  
);  
  
insert into matrix_cases (name, dimension, matrix, determinant)  
values (  
  'tridiagonal-positive-definite',  
  3,  
  '[[2, -1, 0], [-1, 2, -1], [0, -1, 2]]'::jsonb,  
  4  
);
```

第 6 页：概率分布、统计推断与 JavaScript

正态总体抽样中，若 $X_1, X_2, \dots, X_n \sim N(\mu, \sigma^2)$ ，则

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1), \quad \frac{(n-1)S^2}{\sigma^2} \sim \chi^2(n-1).$$

当 σ^2 未知时，有

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t(n-1).$$

分布	统计量	用途
标准正态	$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}}$	方差已知
t 分布	$T = \frac{\bar{X} - \mu}{S/\sqrt{n}}$	方差未知
卡方分布	$\frac{(n-1)S^2}{\sigma^2}$	方差推断

```
function mean(xs) {
  return xs.reduce((a, b) => a + b, 0) / xs.length;
}

function sampleVariance(xs) {
  const m = mean(xs);
  const sq = xs.map(x => (x - m) ** 2).reduce((a, b) => a + b, 0);
  return sq / (xs.length - 1);
}

const data = [72, 75, 71, 79, 77, 74, 76, 78];
console.log({ mean: mean(data), variance: sampleVariance(data) });
```

第 7 页：微分方程、递推与 Bash

一阶线性微分方程

$$y' + p(x)y = q(x)$$

的通解为

$$y = e^{-\int p(x) dx} \left(\int q(x) e^{\int p(x) dx} dx + C \right).$$

二阶常系数微分方程

$$y'' - 3y' + 2y = e^x$$

对应齐次特征方程 $r^2 - 3r + 2 = 0$ ，特征根为 $r_1 = 1$ 、 $r_2 = 2$ 。因为右端 e^x 与齐次解发生撞根，特解应设为 $y^* = Axe^x$ 。

```
#!/usr/bin/env bash
set -euo pipefail

INPUT=${1:-notes.md}
OUTPUT=${2:-dist/notes.pdf}
THEME=${PDF_THEME:-chatgpt-light}

echo "Input:  $INPUT"
echo "Output: $OUTPUT"
echo "Theme:   $THEME"
node scripts/build-pdf.mjs "$INPUT" "$OUTPUT" --theme "$THEME"
```

第 8 页：算法复杂度、递归树与 Go

归并排序递推式为

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

由主定理可得 $T(n) = O(n \log n)$ 。二分查找递推式为

$$T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n).$$

```
package main

import "fmt"

func BinarySearch(nums []int, target int) int {
    left, right := 0, len(nums)-1
    for left <= right {
        mid := left + (right-left)/2
        if nums[mid] == target {
            return mid
        }
        if nums[mid] < target {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }
    return -1
}

func main() {
    nums := []int{1, 3, 5, 7, 9, 11}
    fmt.Println(BinarySearch(nums, 7))
}
```


第 9 页：前后端协同、状态机与 YAML

任务状态机可表示为

$$queued \xrightarrow{dispatch} running \xrightarrow{ok} succeeded, \quad running \xrightarrow{error} failed \xrightarrow{retry} queued.$$

为了保证幂等性，可以设任务唯一键 $k = (workflow_id, step_id, input_hash)$ 。若数据库中已存在相同 k 且状态为 *succeeded*，则云函数直接返回历史结果。

```
name: Build PDF Regression
on:
  push:
    paths:
      - 'inbox/**/manifest.yml'
      - 'inbox/**/md/**'
      - 'themes/**'
      - 'style.css'

jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v4
        with:
          node-version: 24
      - run: npm install
      - run: npm run ci:build
```

测试点

这一页用于同时检查箭头公式、下划线变量、代码块缩进、YAML 高亮、中文 Callout 标题和普通段落间距。

第 10 页：综合长公式、接口代码与结论

最后测试一个更长的优化问题。给定训练集 $\{(x_i, y_i)\}_{i=1}^m$ ，岭回归目标函数为

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (w^T x_i - y_i)^2 + \frac{\lambda}{2} \|w\|_2^2.$$

其梯度为

$$\nabla J(w) = \frac{1}{m} \sum_{i=1}^m (w^T x_i - y_i) x_i + \lambda w.$$

```
interface ApiResponse<T> {
  ok: boolean;
  requestId: string;
  data?: T;
  error?: { code: string; message: string };
}

async function requestJson<T>(url: string, init?: RequestInit):
Promise<ApiResponse<T>> {
  const requestId = crypto.randomUUID();
  try {
    const res = await fetch(url, {
      ...init,
      headers: { 'x-request-id': requestId, ...(init?.headers ?? {}) },
    });
    const data = await res.json() as T;
    return { ok: res.ok, requestId, data };
  } catch (err) {
    return {
      ok: false,
      requestId,
      error: { code: 'NETWORK_ERROR', message: String(err) },
    };
  }
}
```

最终检查清单

- ☒ 行内公式与中文混排
- ☒ 块级多行公式
- ☒ 矩阵、行列式、范数、极限、积分、级数
- ☒ 表格中的公式
- ☒ 多语言代码块高亮
- ☒ 显式分页与 PDF 书签